

NSAPDEV Final Project Report

NSAPDEV — Server Application Development
2nd Term AY 2025–2026

Project Title: Concurrent Syslog Analytics Server ("Mini-Splunk")

Team Members: Joshua Benedict B. Co and Reyvin Matthew T. Tan

Section: S12B / S31

Filenames: `server_cotan15.py` and `client_cotan15.py`

1. Project Overview & Objectives

1.1 System Summary

Mini-Splunk is a lightweight, centralized log management and analytics server built as a simplified analogue of enterprise SIEM tools such as Splunk. The system ingests raw syslog files streamed over TCP from one or more concurrent CLI clients, parses each log line into structured fields using compiled regular expressions conforming to both RFC 3164 (BSD Syslog) and RFC 5424 (modern syslog), stores parsed entries in a thread-safe in-memory list, and exposes real-time search, count, and administrative commands. It consists of two Python programs: **server_cotan15.py** (the Indexer) and **client_cotan15.py** (the Forwarder).

1.2 Learning Outcomes Achieved

- **CLO1 — Architecture Modelling:** The system follows a Client–Server model. The server is decomposed into five logically labeled modules in the source code (Data Storage, Parsing, Query Engine, Connection Handler, Network), directly reflecting the Separation of Concerns principle. The client is a stateless Forwarder that serializes user commands into a pipe-delimited TCP protocol and renders server responses.
- **CLO2 — OS-Level Threading:** The server uses Python's `threading.Thread` to spawn a dedicated daemon worker thread for every accepted TCP connection. Each thread independently handles `recv_message()`, `process_command()`, and `send_response()` without blocking the main accept loop.
- **CLO3 & CLO4 — Concurrency & Mutual Exclusion:** A `threading.RLock()` named `store_lock` guards the shared `log_store` list. All write operations (`append_logs()` for INGEST, `purge_logs()` for PURGE) hold the lock exclusively. All read operations call `get_snapshot()`, which holds the lock only long enough to copy the list, then releases it immediately so that filtering runs on a local copy with no lock held, minimising contention.

2. Final System Architecture

2.1 Architectural Adjustments from the Week 8 Design Document

Three substantive changes were made between the Week 8 Architecture Document and the final submitted implementation. Each is described below alongside its rationale.

Change 1: Dual-Format Syslog Parsing (RFC 3164 + RFC 5424)

Architecture document: specified a single `SYSLOG_REGEX` targeting RFC 3164 (BSD Syslog) exclusively. The document listed five named groups: `PRI`, `TIMESTAMP`, `HOSTNAME`, `DAEMON/TAG`, and `MSG`.

Final implementation: the Parsing Module compiles *two* separate regular expressions: `RFC3164_REGEX` for BSD syslog and `RFC5424_REGEX` for modern syslog. `parse_line()` attempts RFC 5424 first; if that fails, it tries RFC 3164; lines matching neither are silently discarded by returning `None`.

Rationale: Real-world syslog streams commonly mix both formats. Supporting RFC 5424 increases compatibility and correctness without breaking RFC 3164 ingestion.

Change 2: Expanded Log Entry Schema (5 fields → 8 fields)

Architecture document: defined five string-valued fields per log entry: `timestamp`, `hostname`, `daemon`, `severity`, `message`.

Final implementation: each parsed entry dictionary contains eight fields. Three additional fields are populated by `parse_line()`:

- `"format"`: the string `"RFC3164"` or `"RFC5424"`, recording which parser matched the line.
- `"structured_data"`: the RFC 5424 STRUCTURED-DATA element string (e.g. `[exampleSDID@32473 iut="3"]`), or `None` for RFC 3164 entries.
- `"raw"`: the original unparsed log line, preserved verbatim for auditability.

Change 3: Paginated JSON Query Responses

Architecture document: described `format_entries()` as returning a plain formatted string of matching log lines, or the literal string `"NO_RESULTS"` if empty. No pagination was specified.

Final implementation: all `SEARCH` queries are processed by `format_paginated_response()`, which slices results into pages of `RESULTS_PER_PAGE = 100` entries and returns a JSON-encoded object. `COUNT_KEYWORD` is the only query that returns a plain integer string rather than JSON. The client's `display_paginated_results()` deserializes the JSON, renders the current page, and presents `[N]ext`, `[P]revious`, and `[E]xit` navigation options. If `total_pages <= 1` the loop exits immediately with no user prompt.

The JSON response structure returned by the server for every SEARCH query is:

```
{
  "metadata": {
    "total_results": <int>,    // total matching entries
    "page": <int>,           // current page number (1-indexed)
    "per_page": 100,         // entries per page (RESULTS_PER_PAGE)
    "total_pages": <int>,    // ceil(total_results / per_page)
    "has_next": <bool>,     // page < total_pages
    "has_prev": <bool>,     // page > 1
    "results_on_page": <int> // len of current page slice
  },
  "logs": "<formatted log lines joined by \n>" | "NO_RESULTS"
}
```

2.2 Deviations from the Project Specification

The following behaviors in the final implementation differ from the examples or descriptions in the Project Specification document:

Specification Description	Actual Implementation Behavior
SEARCH_SEVERITY accepts levels such as ERROR and WARN.	SEVERITY_MAP uses ERR (not ERROR) and WARNING (not WARN). Exact valid levels are: EMERG, ALERT, CRIT, ERR, WARNING, NOTICE, INFO, DEBUG. Passing ERROR or WARN returns no results.
PURGE response: "SUCCESS: N indexed log entries have been erased." (includes count).	Actual response is the fixed string: "SUCCESS: Log store purged. All entries have been removed." No count is included; purge_logs() calls log_store.clear() without recording the prior length.
INGEST shows upload progress: "[System Message] Uploading syslog (2.1 MB)...".	Client prints only "[SYSTEM] Connecting to <host>:<port>..." before upload. No file size is displayed.
Search responses show numbered entries: "1. Feb 22 00:05:38 SYSSVR1 systemd[1]:...".	format_entries() outputs entries as "[<timestamp>] <hostname> <daemon> [<severity>] <message>" — no line numbers, and severity is included.
SEARCH_DAEMON performs exact match (searching "sshd" returns only entries for "sshd").	search_by_daemon() uses partial/substring matching: needle in e["daemon"].strip().lower(). This is intentional and documented in the function docstring: "searches like 'nginx' will match daemon names that contain the same value."
Response headers like "Found N matching entries for host 'X':" are shown.	No such header is rendered. The client renders a pagination header: "Query Results: <type> Param: <param>" and "Total Results: N Page P/T Showing K entries".

2.3 Component Overview

The system follows a Client–Server model with a multithreaded backend. The two primary programs are:

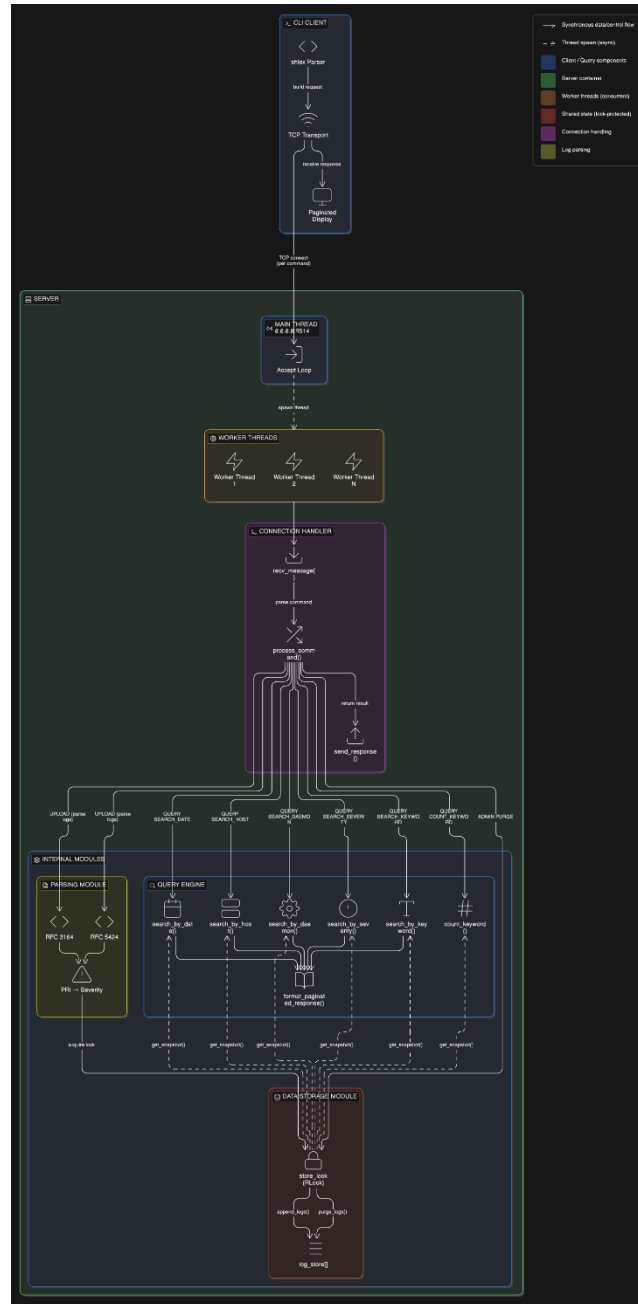
- **client_cotan15.py — The Forwarder:** A stateless CLI client. Presents a `mini-splunk>` prompt, parses user input with `shlex.split()`, opens a new TCP connection per command via `send_simple_command()` or `cmd_ingest()`, and renders server responses via `display_paginated_results()`. Never stores log data locally. Constants: `SERVER_HOST = "103.231.240.136"`, `SERVER_PORT = 11334`.
- **server_cotan15.py — The Indexer:** Binds to `0.0.0.0:8080` with a listen backlog of 10. Accepts connections on the main thread and dispatches each to a daemon worker thread. Maintains all shared state in `log_store`. Constants: `HOST = "0.0.0.0"`, `PORT = 9514`, `RESULTS_PER_PAGE = 100`.

The server source code is organized into five labeled module sections:

Module (as labeled in source)	Key Functions / Variables	Arch. Doc. Grouping
Data Storage Module	<code>log_store</code> , <code>store_lock</code> , <code>append_logs()</code> , <code>purge_logs()</code> , <code>get_snapshot()</code>	Data Storage Module
Parsing Module	<code>RFC3164_REGEX</code> , <code>RFC5424_REGEX</code> , <code>SEVERITY_MAP</code> , <code>SEVERITY_KEYWORDS</code> , <code>parse_pri()</code> , <code>parse_line()</code> , <code>parse_stream()</code> , <code>infer_severity_from_message()</code>	Parsing Module
Query Engine Module	<code>format_entries()</code> , <code>normalize_whitespace()</code> , <code>format_paginated_response()</code> , <code>search_by_date/host/daemon/severity/keyword()</code> , <code>count_keyword()</code>	Query Engine Module
Connection Handler	<code>recv_message()</code> , <code>send_response()</code> , <code>process_upload()</code> , <code>process_query()</code> , <code>process_admin()</code> , <code>process_command()</code> , <code>handle_client()</code>	Network Module (combined)
Network Module	<code>start_server()</code>	Network Module (combined)

Note: The Week 8 Architecture Document described four modules, grouping the Connection Handler functions (`recv_message`, `send_response`, `handle_client`, `process_command`) together with the Network Module. The final source code separates these into two clearly labeled sections for clarity.

2.4 Component Diagram



This architecture outlines a multi-threaded log management system where a CLI client communicates with a central server via TCP transport. The server utilizes a main thread to spawn concurrent workers that route requests through a connection handler for command processing. Logic is split between a parsing module for RFC-compliant log ingestion and a query engine designed for metadata-based searches and counts. All data operations are centralized in a storage module that employs a read-write lock to maintain thread safety during concurrent access. Finally, the system formats and returns paginated results to the client, ensuring an efficient and organized data flow.

3. Concurrency & Synchronization Implementation

3.1 Thread-per-Connection Model

The server uses a **Thread-per-Connection** concurrency model. The main thread runs an infinite `socket.accept()` loop inside `start_server()`. Each accepted connection is immediately handed to a new thread:

```
worker = threading.Thread(  
    target=handle_client,  
    args=(conn, addr),  
    daemon=True,  
)  
worker.start()
```

The main thread never waits for a worker to finish; it loops back to `socket.accept()` immediately. Setting `daemon=True` ensures all worker threads are automatically reclaimed when the server process exits. After each `worker.start()`, the server prints the current active worker count via `threading.active_count() - 1` (subtracting the main thread).

Each worker thread follows this strict lifecycle:

- `recv_message(conn)` — reads one complete command from the socket.
- `process_command(message)` — routes to the correct handler (UPLOAD, QUERY, or ADMIN).
- `send_response(conn, result)` — sends the length-prefixed response.
- `conn.close()` — runs in the `finally` block, guaranteeing socket release even on exceptions.

3.2 Critical Section & Locking Mechanism

The global `log_store = []` is a Critical Section: multiple worker threads may concurrently attempt to append entries (INGEST), clear all entries (PURGE), or read entries (QUERY). The synchronization object is:

```
store_lock = threading.RLock()
```

A **Reentrant Lock** (`RLock`) is used rather than a plain `Lock` because it permits the same thread to acquire the lock multiple times without deadlocking. All access to `log_store` is encapsulated in three functions; no other code touches the list directly:

Function	Lock Behavior	Called By
<code>append_logs(entries)</code>	with <code>store_lock</code> : <code>log_store.extend(entries)</code> — exclusive write lock held for the entire extend.	<code>process_upload()</code> (INGEST)
<code>purge_logs()</code>	with <code>store_lock</code> : <code>log_store.clear()</code> — exclusive write lock held for the entire clear.	<code>process_admin()</code> (PURGE)
<code>get_snapshot()</code>	with <code>store_lock</code> : <code>return list(log_store)</code> — lock held only during the shallow copy; released before filtering begins.	All six SEARCH/COUNT functions

The snapshot pattern in `get_snapshot()` is the key design decision: once a query thread holds its local copy, any concurrent INGEST or PURGE cannot affect the query's result. This means query threads can run their list-comprehension filters entirely without holding the lock, maximising read parallelism.

4. Communication Protocol & Parsing

4.1 Wire Protocol

All communication occurs over raw TCP on port **8080**. The protocol is **text-based and pipe-delimited (|)**. The client opens a **new TCP connection per command** and closes it after receiving the response.

Command	Bytes Sent by Client
INGEST	UPLOAD <filesize> \n followed immediately by exactly <filesize> raw UTF-8 bytes of syslog content.
QUERY (page 1)	QUERY <QUERY_TYPE> <param>\n (e.g. QUERY SEARCH_HOST SYSSVR1\n)
QUERY (page N>1)	QUERY <QUERY_TYPE> <param> <page>\n (e.g. QUERY SEARCH_DATE Feb 22 2\n)
COUNT_KEYWORD	QUERY COUNT_KEYWORD <word>\n
PURGE	ADMIN PURGE\n

Length-prefix framing: Every server response is framed as <byte_length>\n<response_body>. `send_response()` encodes the body to UTF-8, prepends the byte count and a newline, then calls `conn.sendall()` twice. The client's `recv_response()` reads the integer prefix byte-by-byte until the newline, then reads exactly that many bytes to reconstruct the complete payload regardless of embedded newlines in multi-line query results.

Server-side command splitting: The command type is detected via `message.split("|", 1)[0].upper()`. Splitting limits differ by type to prevent pipe characters inside log content from corrupting parsing: UPLOAD splits at most 2 pipes (`split("|", 2)`), QUERY at most 3 (`split("|", 3)`), ADMIN at most 1 (`split("|", 1)`).

4.2 Data Ingestion Flow

When the client issues INGEST, `cmd_ingest()` reads the local file entirely into memory with `open(filepath, "r", encoding="utf-8")`, encodes it to UTF-8 bytes, and sends a two-part message: the header `f"UPLOAD|{filesize}|\n"` followed immediately by the raw content bytes. On the server side, `recv_message()` performs a two-phase receive:

Phase 1 — header: reads bytes one at a time until a newline, producing the header string (e.g. `UPLOAD|4096|`).

Phase 2 — body: parses the filesize from the header (`header_str[7 : header_str.index("|", 7)]`), then reads up to `min(4096, remaining)` bytes per iteration until `len(content_buf) == filesize`.

Once fully received, `process_upload()` passes the content to `parse_stream()` and calls `append_logs(entries)`. The server response is: `f"SUCCESS: Ingested {len(entries)} log entries into the store"`.

4.3 Syslog Parsing Engine

The Parsing Module compiles two regular expressions at import time. `parse_line()` strips trailing carriage returns and newlines, skips blank lines, then attempts RFC 5424 first, falling back to RFC 3164. The severity is computed by `parse_pri()` as `int(pri_field) & 0x07`, then looked up in `SEVERITY_MAP`. If the PRI field is absent (RFC 3164 allows this), severity is inferred from the message content using predefined keywords (`SEVERITY_KEYWORDS`), and defaults to "INFO" if no keyword matches.

RFC 3164 Regular Expression (RFC3164_REGEX):

```
re.compile(
    r"^\s*"
    r"(?:<(?P<pri>\d{1,3}>)\s*)?"
    r"(?P<timestamp>"
        r"(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec) "
        r"\s{1,2}"
        r"\d{1,2}"
        r"\s+"
        r"\d{2}:\d{2}:\d{2}"
    r")\s+"
    r"(?P<hostname>\S+)\s+"
    r"(?P<tag>[^\s:\[\]]+)"
    r"(?:\[ (?P<pid>\d+) \])?"
    r":\s*"
    r"(?P<msg>.*) $"
)
```

RFC 5424 Regular Expression (RFC5424_REGEX):

```
re.compile(
    r"^\s*"
    r"<(?P<pri>\d{1,3})>"
    r"(?P<version>[1-9]\d{0,2})\s+"
    r"(?P<timestamp>"
        r"-|"
        r"\d{4}-\d{2}-\d{2}T"
        r"\d{2}:\d{2}:\d{2}"
        r"(?:\.\d{1,6})?"
        r"(?:Z| [+ -]\d{2}:\d{2})"
    r")\s+"
    r"(?P<hostname>\S+)\s+"
    r"(?P<appname>\S+)\s+"
    r"(?P<procid>\S+)\s+"
    r"(?P<msgid>\S+)\s+"
    r"(?P<structured_data>"
        r"-|"
        r"(?:\[ (?:\.|\.|\[^\]])*\])+"
    r")"
    r"(?:\s+(?P<msg>.*))?$"
)
```

Log entry schema — all eight fields stored per parsed line:

Field	Type	Source	Example Value
timestamp	str	RFC3164 TIMESTAMP group / RFC5424 TIMESTAMP group	"Feb 22 00:05:38" or "2025-02-22T00:05:38Z"
hostname	str	RFC3164 HOSTNAME group / RFC5424 HOSTNAME group	"SYSSVR1"
daemon	str	RFC3164 TAG group (PID stripped) / RFC5424 APPNAME group	"systemd" or "sshd"
severity	str	parse_pri(pri_group) or infer_severity_from_message(msg)	"INFO", "ERR", "WARNING", "DEBUG"
message	str	RFC3164 MSG group / RFC5424 MSG group (empty string if absent)	"Started OpenBSD Secure Shell server"
format	str	Set by parse_line() based on which regex matched	"RFC3164" or "RFC5424"
structured_data	str None	RFC5424 STRUCTURED-DATA group; None for RFC3164 entries	"-" or "[exampleSDID@32473 iut=\"3\"]"
raw	str	Original line after rstrip("\n\r")	"<134>Feb 22 00:05:38 SYSSVR1 systemd: Started..."

SEVERITY_MAP — PRI & 0x07 mapping:

PRI & 0x07	SEVERITY_MAP value
0	EMERG
1	ALERT
2	CRIT
3	ERR
4	WARNING
5	NOTICE
6	INFO
7	DEBUG

Note on SEARCH_SEVERITY: Valid query values are exactly the strings in SEVERITY_MAP: EMERG, ALERT, CRIT, ERR, WARNING, NOTICE, INFO, DEBUG. The specification's example values ERROR and WARN will return zero results because they do not appear in SEVERITY_MAP.

4.4 Query Engine Match Strategies

Each search function uses a distinct matching strategy as implemented in the source code:

Query Type	Match Strategy	Implementation Detail
SEARCH_DATE	Substring, case-insensitive, whitespace-normalized	normalize_whitespace(needle) in normalize_whitespace(e["timestamp"]). Both sides collapse repeated spaces to a single space and convert to lowercase before comparison.
SEARCH_HOST	Exact, case-insensitive	e["hostname"].strip().lower() == needle.strip().lower()
SEARCH_DAEMON	Substring, case-insensitive (partial match)	needle in e["daemon"].strip().lower(). Intentional per docstring: allows "nginx" to match "nginx-worker".
SEARCH_SEVERITY	Exact, case-insensitive (uppercased before compare)	e["severity"].strip().upper() == level.strip().upper()
SEARCH_KEYWORD	Substring, case-insensitive	needle in e["message"].lower()
COUNT_KEYWORD	Substring, case-insensitive (count only)	sum(1 for e in snapshot if needle in e["message"].lower()). Returns a plain integer string, not JSON.

5. System User Manual

5.1 Prerequisites

- **Python 3.8 or higher** — no third-party packages required. Standard library only: `socket`, `threading`, `re`, `json`, `os`, `shlex`, `io`.
- Both `server_cotan15.py` and `client_cotan15.py` available on the machine(s) to be used.
- TCP port **8080** open and not blocked by a firewall or OS-level port restriction.

5.2 Starting the Server

Open a terminal and run:

```
python server_cotan15.py
```

The server prints the following exact startup banner and then blocks on `socket.accept()`:

```
=====
Mini-Splunk Concurrent Syslog Analytics Server
Listening on 0.0.0.0:8080
Press Ctrl+C to shut down
=====
```

The server binds to `0.0.0.0` (all interfaces) on port `8080` with `SO_REUSEADDR` set. To stop the server, press **Ctrl+C**; it prints a shutdown message and closes the socket cleanly.

Each time a client connects, the server prints: `[CONNECT] <IP>:<port>`, then `[THREADS] Active worker threads: N`, then `[COMMAND] <IP>:<port> -> <first 80 chars of message>`, then `[RESPONSE] <IP>:<port> <- <first 80 chars of response>`, then `[DISCONNECT] <IP>:<port>`.

5.3 Starting the CLI Client

Open a second terminal and run:

```
python client_cotan15.py
```

Startup banner:

```
=====
Mini-Splunk CLI Forwarder
Target server: 103.231.240.136:11334
Type 'help' for available commands or 'exit' to quit.
=====

mini-splunk>
```

The client uses `shlex.split()` to tokenize input, so multi-word query values must be enclosed in double quotes (e.g. "Feb 22" or "Failed password"). Type `HELP` to display the built-in command reference. Type `EXIT` to quit.

5.4 Command Reference

Note on Commands: In all examples below, `<IP_or_DNS>:<Port>` refers to the address and port of the running server. The default is `103.231.240.136:11334` as hardcoded in `client_cotan15.py`.

A. INGEST — Upload a Syslog File

- **Format:** `INGEST <filepath> <IP_or_DNS>:<Port>`
- **Behavior:** Opens the file at `<filepath>` in UTF-8 read mode, encodes it to bytes, and streams it to the server. The server parses and indexes the entries.
- **Example input/output:**

```
mini-splunk> INGEST /var/log/syslog <IP_or_DNS>:<Port>
[SYSTEM] Connecting to <IP_or_DNS>:<Port>...
[INGEST]          SUCCESS: Ingested 1842 log entries into the store
```

- **File not found:** prints `[ERROR] File not found: <filepath>` and returns immediately without connecting.

B. SEARCH_DATE — Filter by Timestamp

- **Format:** `QUERY <IP_or_DNS>:<Port> SEARCH_DATE "<date_string>"`
- **Match strategy:** substring, case-insensitive, whitespace-normalized (see Section 4.4).
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> SEARCH_DATE "Feb 22"
[System Message] Sending query to <IP_or_DNS>:<Port>...
=====
Query Results: SEARCH_DATE | Param: Feb 22
=====
```

```
Total Results: 312 | Page 1/4 | Showing 100 entries
-----
[Feb 22 00:05:38] SYSSVR1 systemd [INFO] Started OpenBSD Secure Shell...
...
-----
Options: [N]ext | [E]xit

Enter command (n/p/e):
```

C. SEARCH_HOST — Filter by Hostname

- **Format:** `QUERY <IP_or_DNS>:<Port> SEARCH_HOST <hostname>`
- **Match strategy:** exact, case-insensitive.
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> SEARCH_HOST SYSSVR1
[System Message] Sending query to <IP_or_DNS>:<Port>...
Total Results: 58 | Page 1/1 | Showing 58 entries
[Feb 22 00:05:38] SYSSVR1 systemd [INFO] Started OpenBSD Secure Shell...
```

D. SEARCH_DAEMON — Filter by Daemon Name

- **Format:** `QUERY <IP_or_DNS>:<Port> SEARCH_DAEMON <daemon_name>`
- **Match strategy:** substring (partial), case-insensitive. A query for `ssh` matches `sshd`, `ssh-agent`, etc.
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> SEARCH_DAEMON sshd
[Feb 22 01:14:22] SYSSVR1 sshd [ERR] Failed password for invalid user
admin
```

E. SEARCH_SEVERITY — Filter by Severity Level

- **Format:** `QUERY <IP_or_DNS>:<Port> SEARCH_SEVERITY <level>`
- **Match strategy:** exact, case-insensitive (uppercased before comparison).
- **Valid levels:** EMERG, ALERT, CRIT, ERR, WARNING, NOTICE, INFO, DEBUG. **Note:** ERROR and WARN are **not** valid and will return no results.
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> SEARCH_SEVERITY ERR
[Feb 22 01:14:22] SYSSVR1 sshd [ERR] Failed password for invalid user
admin
```

F. SEARCH_KEYWORD — Full-Text Search on Message

- **Format:** `QUERY <IP_or_DNS>:<Port> SEARCH_KEYWORD <keyword_or_phrase>`
- **Match strategy:** substring, case-insensitive, against the `message` field only.
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> SEARCH_KEYWORD "Failed password"
[Feb 22 01:14:22] SYSSVR1 sshd [ERR] Failed password for invalid user
admin from 10.0.0.9
```

G. COUNT_KEYWORD — Count Messages Containing a Keyword

- **Format:** `QUERY <IP_or_DNS>:<Port> COUNT_KEYWORD <keyword_or_phrase>`
- **Match strategy:** same as `SEARCH_KEYWORD` but returns an integer count, not log lines. Response is a plain string, not JSON.
- **Example:**

```
mini-splunk> QUERY <IP_or_DNS>:<Port> COUNT_KEYWORD "Failed password"
[COUNT_KEYWORD]   Entries containing 'Failed password': 14
```

H. PURGE — Clear All Indexed Logs

- **Format:** `PURGE <IP_or_DNS>:<Port>`
- **Behavior:** instructs the server to acquire an exclusive write lock via `purge_logs()` and call `log_store.clear()`. All indexed entries are permanently removed from memory. The server response does not include the number of deleted entries.
- **Example:**

```
mini-splunk> PURGE <IP_or_DNS>:<Port>
[SYSTEM] Connecting to <IP_or_DNS>:<Port> to purge records...
[PURGE]      SUCCESS: Log store purged. All entries have been
removed.
```

I. Pagination Navigation

When a `SEARCH` query returns more than **100 entries**, the client presents results page by page. If `total_pages <= 1` the paginator exits immediately without prompting. Otherwise:

- `n` or `N` — advance to the next page (only shown if `has_next == true`)
- `p` or `P` — go back to the previous page (only shown if `has_prev == true`)
- `e` or `E` or `EXIT` — exit the paginated view
- Any other input prints: Invalid command. Use `[N]ext`, `[P]revious`, or `[E]xit`.

Each page request is a fresh TCP connection sending `QUERY|<TYPE>|<param>|<page_number>`.

6. Testing & Performance Evaluation

6.1 Concurrency Testing

The server's concurrent ingestion capability was validated by launching multiple client instances simultaneously, each uploading a syslog file at the same time. The final entry count in `log_store` after all uploads completed always equalled the sum of the individually parsed entry counts, confirming that `threading.RLock()` and `log_store.extend()` together prevent interleaved writes from producing data corruption or entry loss.

Concurrent read-write behavior was also tested: while one client performed an INGEST, a second client issued SEARCH_HOST queries. The `get_snapshot()` pattern ensured that each query operated on a consistent point-in-time copy of the store — either the full pre-ingest state or the full post-ingest state, never a partial intermediate. The server's printed `[THREADS] Active worker threads: N` lines confirmed that multiple threads were active simultaneously.

6.2 Edge Cases Handled

- **Invalid commands (client-side):** The CLI parser validates syntax using `shlex.split()` before sending. Unknown command types print `[ERROR] Unknown command '<cmd>'`. QUERY with fewer than 4 tokens, INGEST with fewer than 3, and PURGE with a count other than 2 all print usage strings and do not connect.
- **Unknown query types (server-side):** `process_query()` returns `f"ERROR: Unknown query type '{query_type}'"` for any QUERY type not in the dispatch table.
- **Malformed syslog lines:** `parse_line()` strips trailing `\r\n`, skips blank lines, and returns `None` for lines matching neither regex. Only entries returning a non-`None` dict are appended to the log store via `parse_stream()`.
- **Empty query results:** `format_paginated_response()` handles zero matches correctly: `total_pages = 0`, page is clamped to 1, and `format_entries([])` returns the string `"NO_RESULTS"`. The client renders `"No results found."` and exits the paginator immediately.
- **Client connection refused:** `send_simple_command()` catches `ConnectionRefusedError` and returns the string `f"ERROR: Connection refused. Is the server running on {host}:{port}?"` without crashing. `cmd_ingest()` catches it separately and prints the same message.
- **Server-side exceptions:** `handle_client()` wraps all operations in `try/except`. On any exception, it attempts to send `f"ERROR: Server exception: {exc}"` to the client, then closes the socket in the `finally` block unconditionally.

- **Malformed length prefix:** If `recv_response()` on the client side receives a non-integer before the newline, it returns `"ERROR: Received malformed length prefix from server"` rather than crashing.
- **Invalid page number in QUERY:** `process_query()` defaults page to 1 if the fourth pipe-segment is absent or non-integer. `format_paginated_response()` further clamps the page to the range `[1, total_pages]` using `max(1, min(page, total_pages))`.
- **PURGE during QUERY:** Because `purge_logs()` holds the exclusive RLock during `clear()`, any concurrent `get_snapshot()` call will block until the purge completes, then return an empty list. This is the correct, safe behavior.

7. Intellectual Honesty Declaration

We, **Joshua Benedict B. Co** and **Reyvin Matthew T. Tan**, declare that the submitted source code (`server_cotan15.py` and `client_cotan15.py`) and this documentation are the product of our own collaborative effort and reflect our commitment to academic integrity.

AI tools, specifically **Claude Sonnet 4.6** and **ChatGPT-5.3**, were utilized solely for programming assistance and report formatting guidance. All AI-generated suggestions were critically reviewed, tested, and modified by the authors to align with the project specifications.

The implementation makes exclusive use of Python's standard library (`socket`, `threading`, `re`, `json`, `os`, `io`, and `shlex`) in full compliance with the project constraints. All referenced works and external documentation sources are properly cited in the Appendix.

8. Appendices

Appendix A — Source Code File Structure

```
mini-splunk/  
  server_cotan15.py      # Indexer (Thread-per-Connection server)  
  client_cotan15.py     # Forwarder (interactive CLI client)
```

Module breakdown within server_cotan15.py (as labeled in source):

Source Label	Functions / Variables
Data Storage Module	log_store store_lock append_logs() purge_logs() get_snapshot()
Parsing Module	SEVERITY_KEYWORDS SEVERITY_MAP RFC3164_REGEX RFC5424_REGEX infer_severity_from_message() parse_pri() parse_line() parse_stream()
Query Engine Module	format_entries() normalize_whitespace() format_paginated_response() search_by_date() search_by_host() search_by_daemon() search_by_severity() search_by_keyword() count_keyword()
Connection Handler	recv_message() send_response() process_upload() process_query() process_admin() process_command() handle_client()
Network Module	start_server()

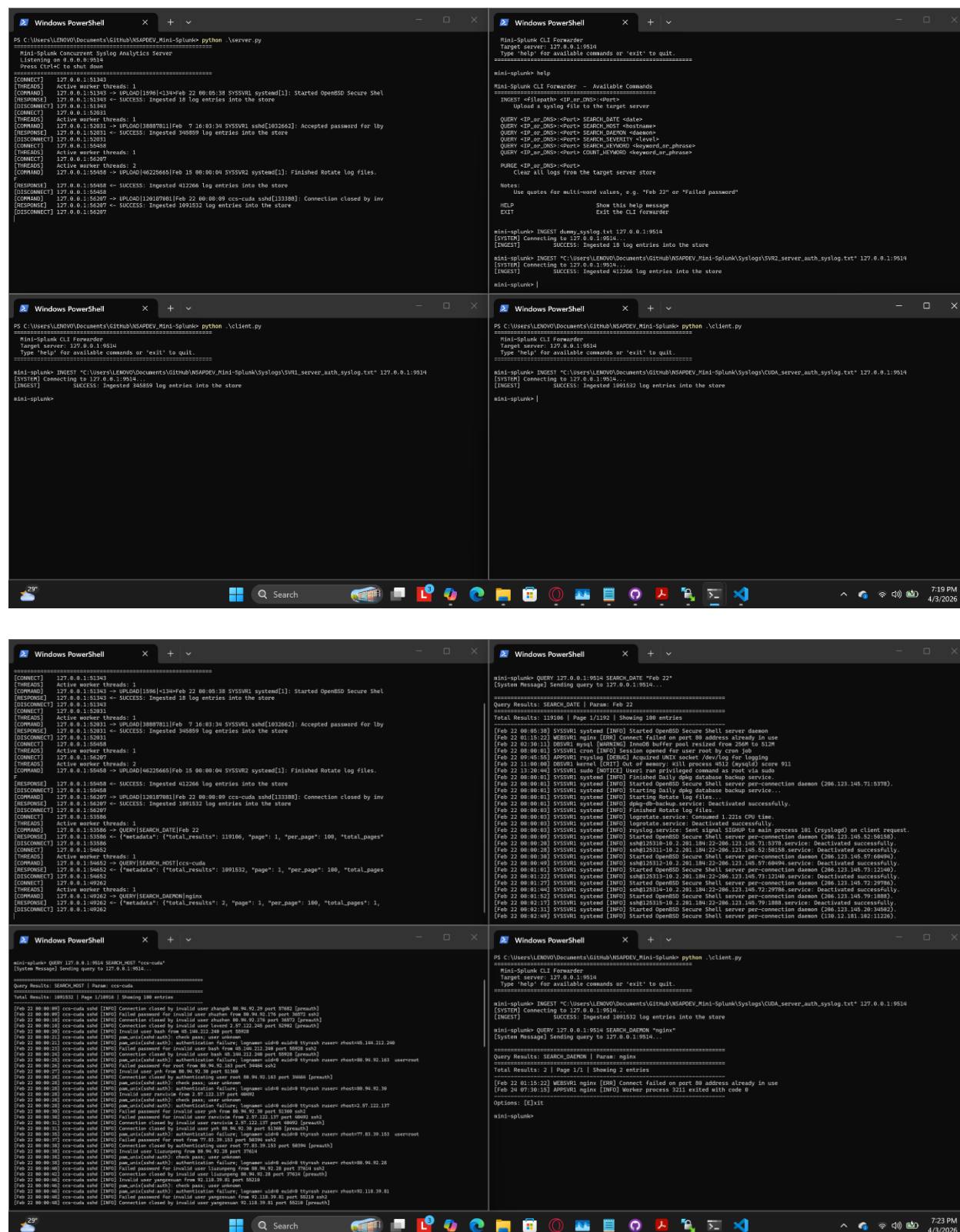
Module breakdown within client_cotan15.py:

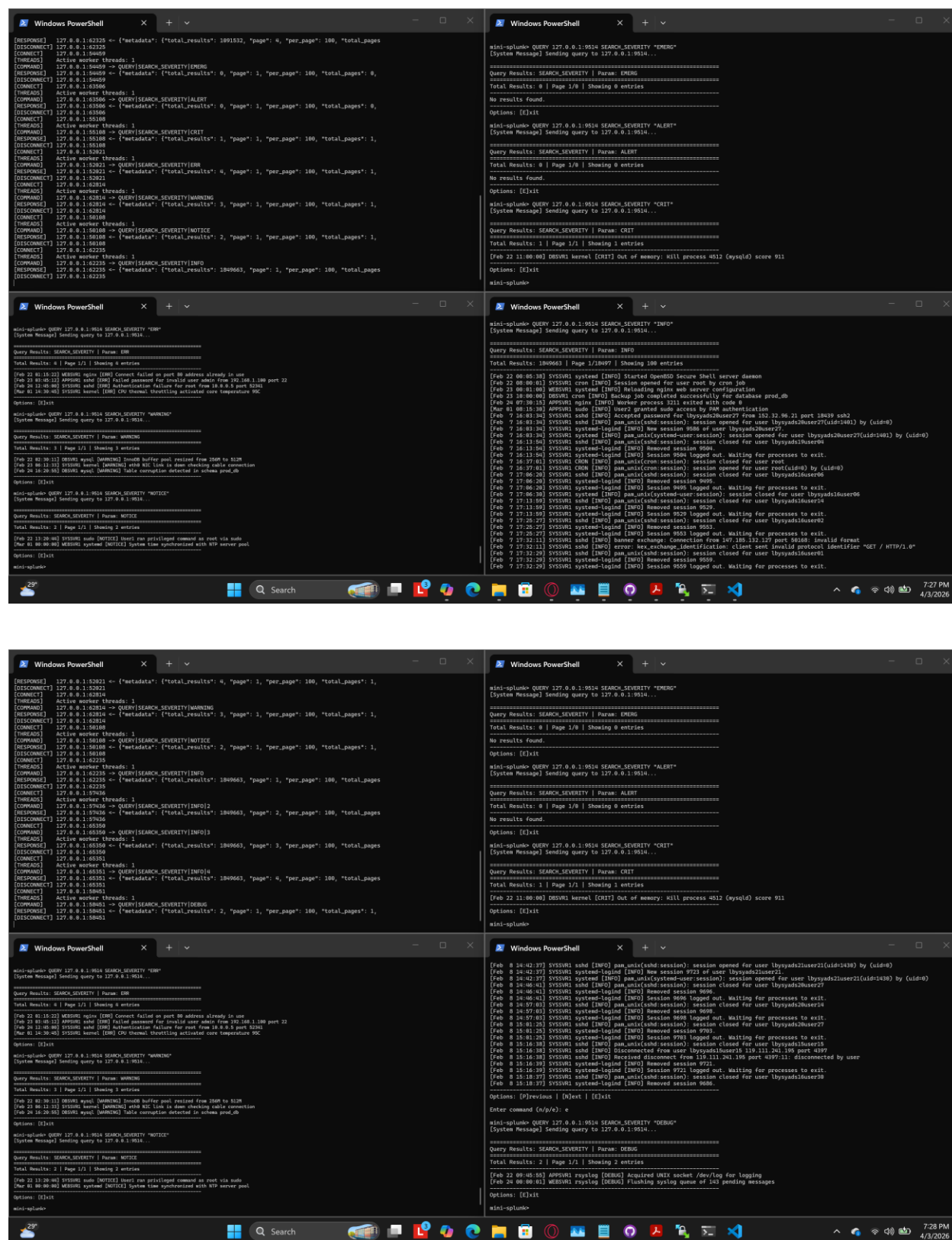
Source Label	Functions / Variables / Constants
Constants	SERVER_HOST = "103.231.240.136" SERVER_PORT = 11334
Transport Helpers	recv_response() parse_endpoint() send_simple_command() display_paginated_results()
Command Implementations	cmd_ingest() cmd_search_date() cmd_search_host() cmd_search_daemon() cmd_search_severity() cmd_search_keyword() cmd_count_keyword() cmd_query() cmd_purge()
CLI Shell	HELP_TEXT QUERY_TYPES main()

Appendix B — GitHub Link

GitHub Link: https://github.com/Reyvin2005/NSAPDEV_Mini-Splunk.git

Appendix C — Group 15 Mini-Splunk Local Environment Testing

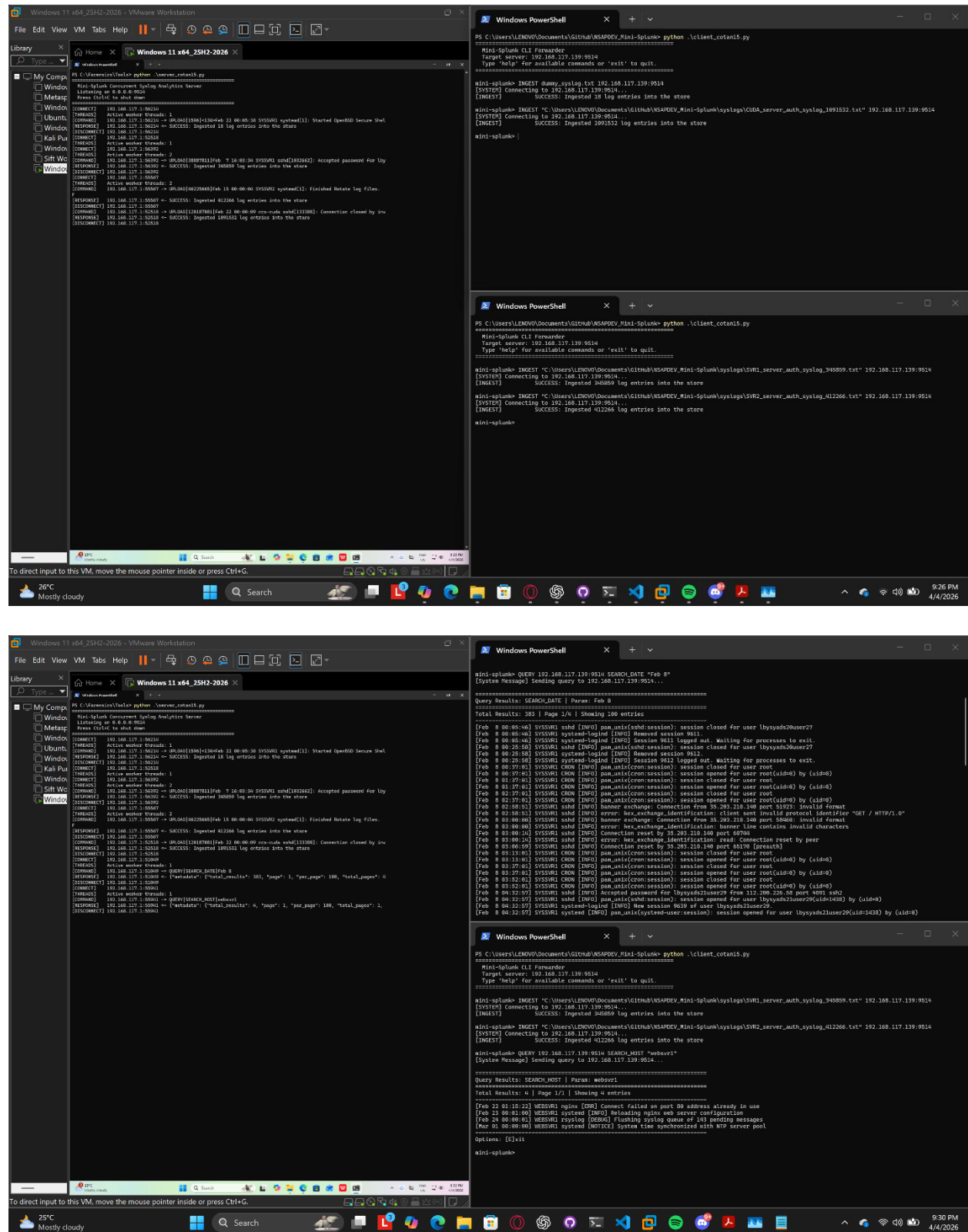




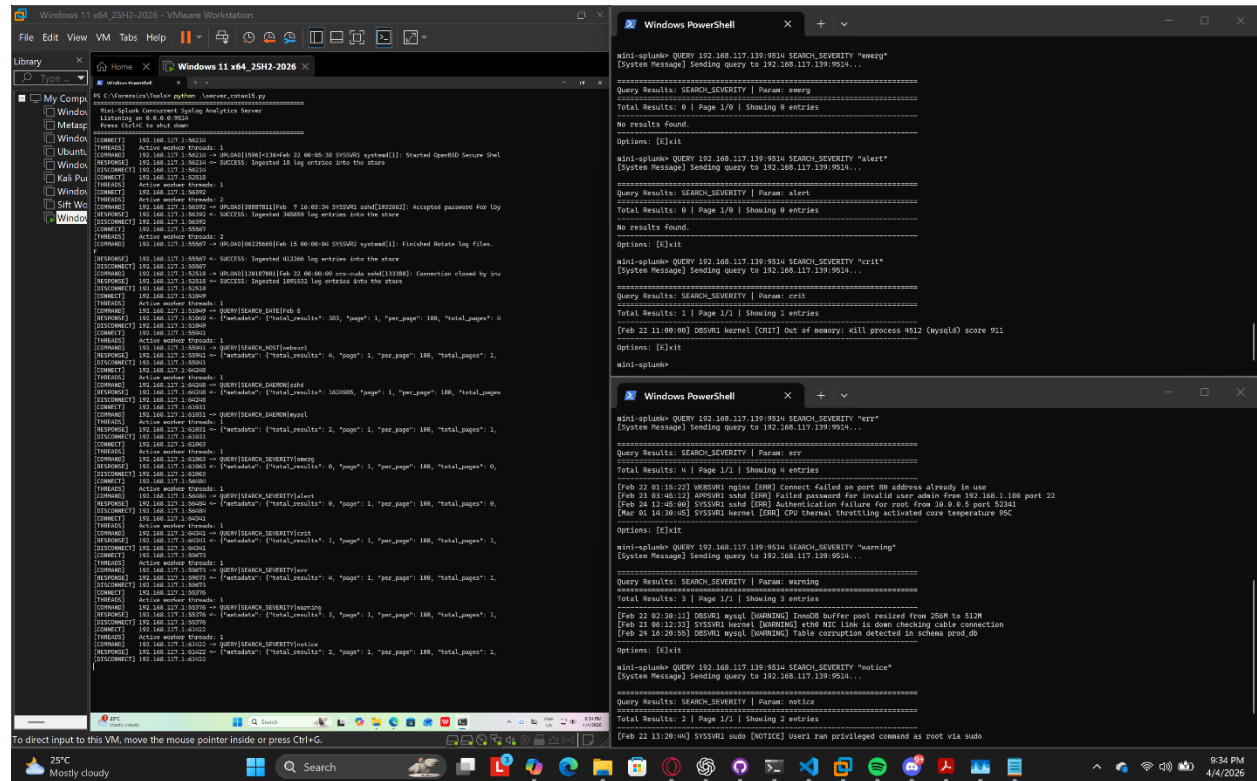
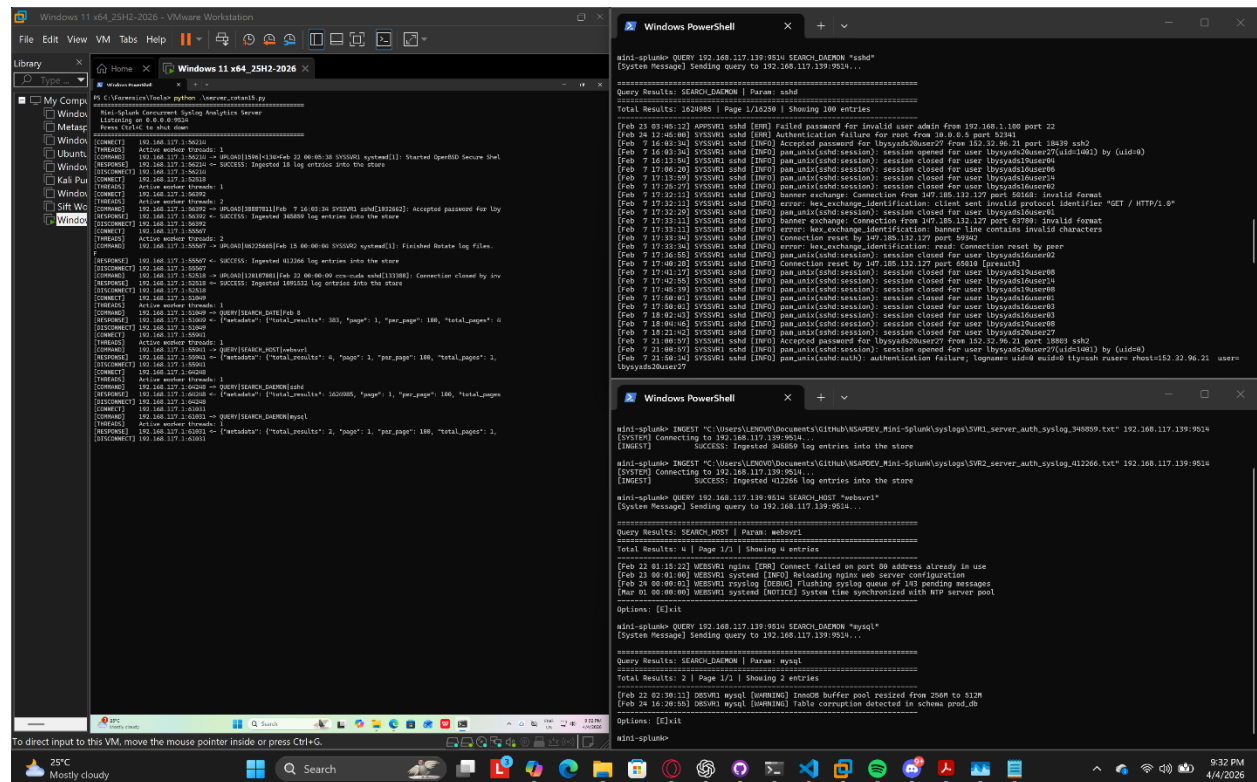
[illegible]

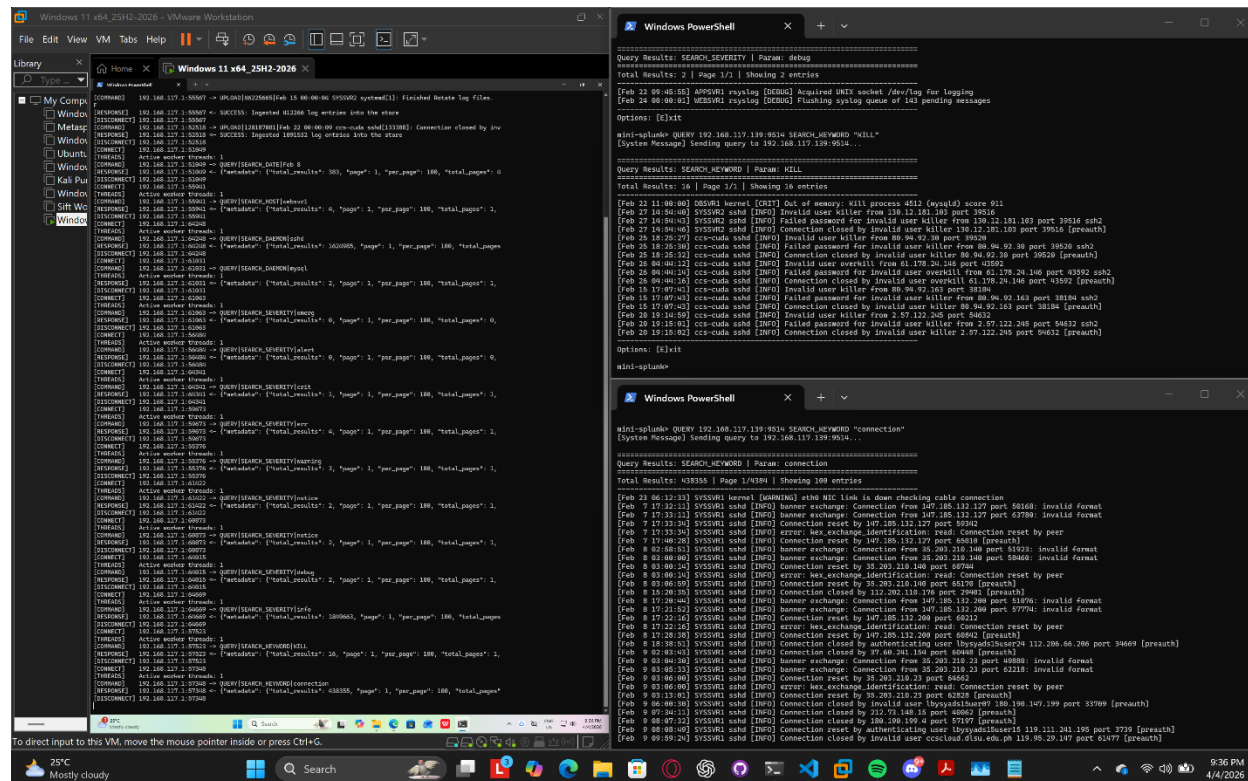
[illegible]

Appendix D — Group 15 Mini-Splunk Network-Based Testing

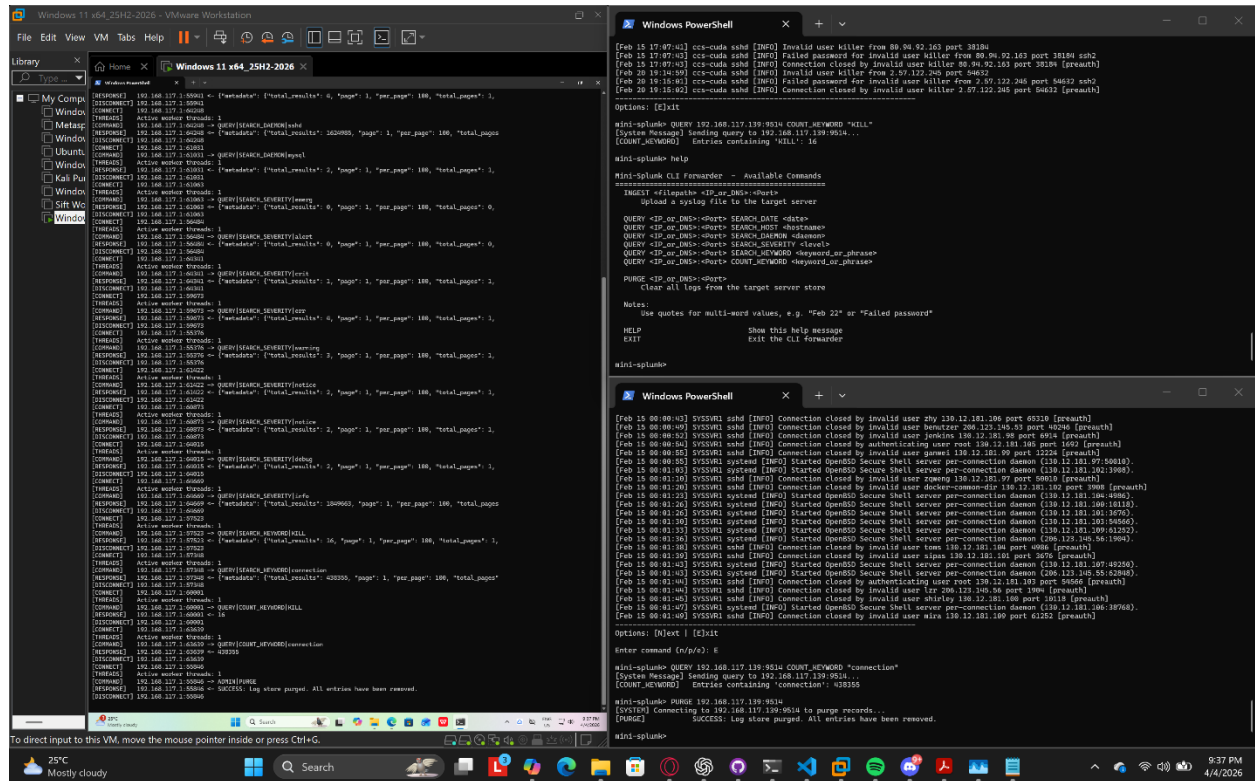
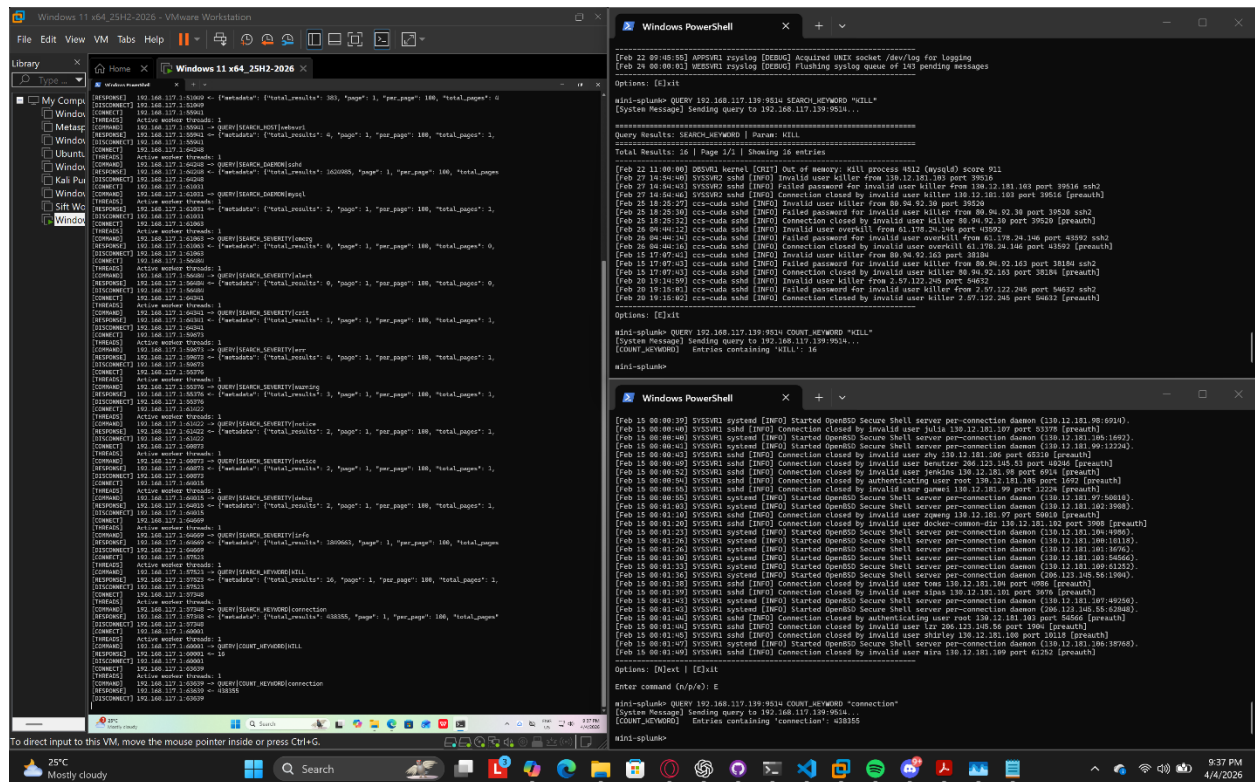


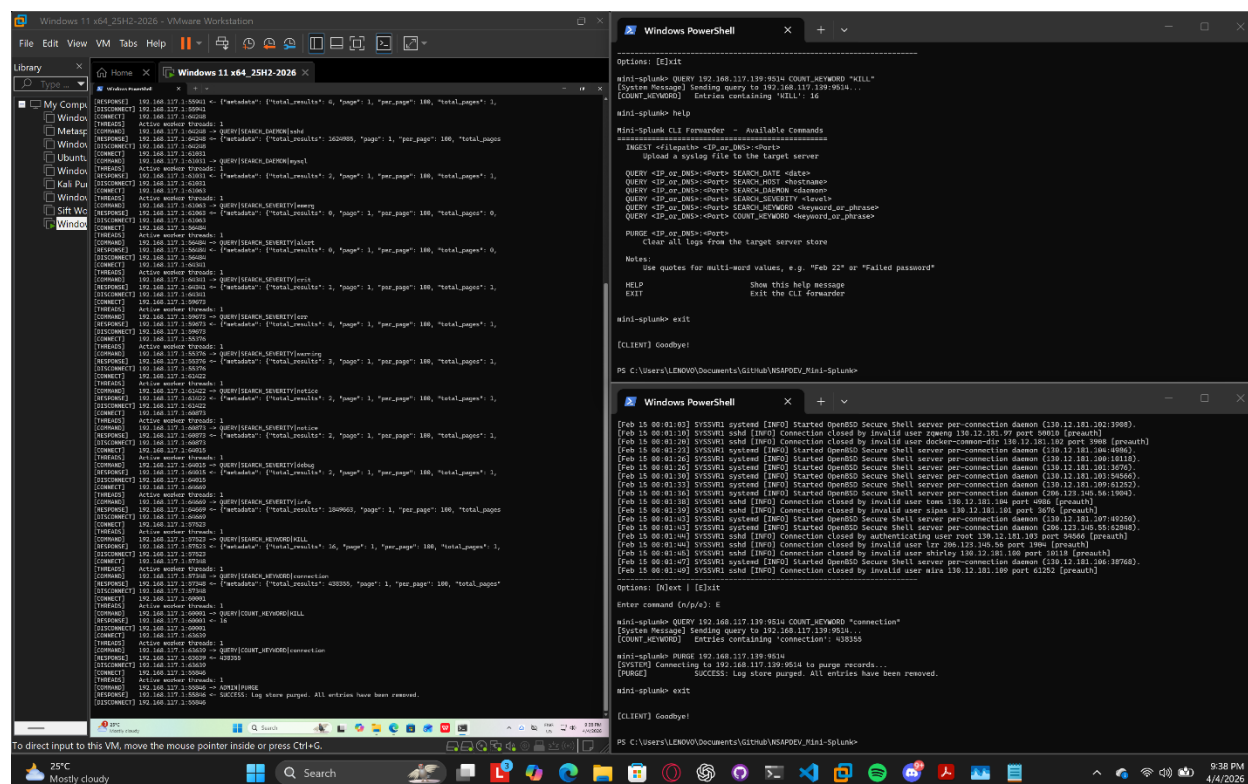
NSAPDEV Final Project Report — Concurrent Syslog Analytics Server ("Mini-Splunk")





NSAPDEV Final Project Report — Concurrent Syslog Analytics Server ("Mini-Splunk")





Appendix E — References

- [1] C. Lonvick, "The BSD Syslog Protocol," RFC 3164, Internet Engineering Task Force, Aug. 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3164>
- [2] R. Gerhards, "The Syslog Protocol," RFC 5424, Internet Engineering Task Force, Mar. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5424>
- [3] Python Software Foundation, "threading — Thread-based parallelism," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [4] Python Software Foundation, "socket — Low-level networking interface," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/socket.html>
- [5] Python Software Foundation, "re — Regular expression operations," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/re.html>
- [6] B. Hall, "Beej's Guide to Network Programming: Using Internet Sockets," ver. 3.1.11, 2023. [Online]. Available: <https://beej.us/guide/bgnet/>
- [7] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ver. 1.10. Arpaci-Dusseau Books, 2023. [Online]. Available: <https://pages.cs.wisc.edu/~remzi/OSTEP/>